Project Model Selection – Mod4

Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

Sanjay Kumar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SanjayKumar@sandiego.edu

Santa Das Karmakar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SDasKarmakar@sandiego.edu

Sethuraman Krishnasamy
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SKrishnasamy@sandiego.edu

Project Model Selection – Mod4 (AI590)

After exploring the Gretel synthetic text-to-SQL dataset and deciding that the central application goal is **building a helpful BigQuery-oriented SQL generation assistant**, the next important decision is **which model architecture and training approach to use**.

Below are the realistic considerations and reasoning that led to the method choice.

Project Requirements:

- Must include **at least one neural network / deep learning model**
- Must include **model training** (from scratch, full fine-tuning, or parameter-efficient fine-tuning)
- Should **not** be just following a public end-to-end tutorial without meaningful original decisions
- Should be feasible with student-level compute budget & time (~1–2 months of part-time work)

Approaches

Approach	Compute needed	Data hunger	Speed to first result	Expected SQL quality	Originality needed	Fits project rules?
Fine-tune small encoder-decoder (T5-small / Flan-T5-base)	low–medium	medium	very fast	acceptable–good	medium	yes
Fine-tune CodeLlama / DeepSeek-Coder 6.7B–13B	high	medium	slow	good–very good	medium	yes
Prompt very large model (zero / few-shot)	almost none	none	instant	medium–good	low	**no** (no training)
Full fine-tuning Llama-3 8B / Mixtral 8x7B	very high	high	very slow	very good	medium	borderline
LoRA / QLoRA on Gemma-2/3 9B–27B or similar	medium–high	medium	acceptable	very good–excellent	high	**yes**
LoRA on very small model (Phi-3-mini, Gemma-2-2B)	low	medium	fast	good	high	yes
Mixture-of-Experts routing on small specialists	very high	high	slow	potentially excellent	very high	yes but risky

Decision Rationale – Why Gemma + LoRA right now

Rationale behind the following choices:

- **Base model**: google/gemma-3-4b-it (most likely a typo / old name → actually Gemma 2 9B or Gemma 3 family 4B instruction-tuned variant)
- **Method**: LoRA (Low-Rank Adaptation)
- **Rank (r)**: 32
- **Target modules**: almost all linear layers in attention + MLP blocks
- **Quantization** friendly setup (paged_adamw_32bit, expandable_segments, etc.)
- **Infrastructure**: GKE + NVIDIA L4 GPU + GCS Fuse + custom training container

Main reasons this combination makes sense:

1. **Instruction-tuned starting point is very valuable for chat-style text-to-SQL** Gemma-2/3-it already understands chat format, system prompts, <think> tags, code blocks → much less prompt engineering pain compared to base models.

2. **LoRA is the best compromise between quality and compute budget**
    - Full fine-tuning of even 9B model is usually unrealistic on one L4 (24 GB)
    - QLoRA + LoRA allows training with 4-bit / 8-bit base weights → fits larger models
3. **Gemma family is relatively permissive for academic/commercial use** (compared to some other 2025–2026 open models)
4. **Good trade-off quality vs. size**
    - 2B → fast but noticeably weaker SQL generation
    - 9B/27B LoRA → strong zero-shot + fine-tuning can reach very good BigQuery SQL level
    - 70B class → usually too expensive / too slow for student project iteration
5. **BigQuery dialect focus is easier to teach with smaller parameter updates** → LoRA on instruction model + synthetic BigQuery-flavored data is a very efficient way

Realistic Alternatives to still Consider (mid-project pivot)

Situation	**Suggested pivot**	**Difficulty to switch**
Model is clearly underfitting even after many epochs	Switch to larger base model (Gemma 2 27B, Llama-3.1 8B)	medium–high
Training is extremely slow / OOM every run	Reduce LoRA rank to 16–24, use 4-bit base, smaller batch	low
Quality is already surprisingly good	Try adding DPO / ORPO preference optimization phase	high
Want strongest possible result, have 2–3 weeks extra	Switch to QLoRA + Gemma 2 27B or Llama-3.1 8B/70B	high
Want faster iteration & cheaper experiments	Temporarily switch to Phi-3.5-mini or Gemma-2-2B	low

Summary – Current Choice Evaluation

Chosen method: LoRA fine-tuning of an instruction-tuned Gemma 3 (≈4B–9B class) model on synthetic BigQuery-flavored text-to-SQL data

Strengths

- Matches "must include deep learning + training" requirement
- High likelihood of producing usable SQL generation quality
- Compute-feasible on single L4 GPU (with QLoRA tricks)
- Allows meaningful original contributions (prompt formatting, schema augmentation, dialect normalization, negative sampling, etc.)

Main risks / things to watch

- If rank=32 + targeting almost all modules → may still run out of memory
- Very short max_length=64 → probably too short for complex SQL queries
- Need to carefully evaluate whether generated SQL actually runs on BigQuery (syntax + semantics)
- Possibility to increase max_length to 256–512 and add proper evaluation metrics — execution accuracy + exact match.